ДИСЦИПЛИНА	Технологии разработки приложений на базе фреймворков
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практическая работа
ПРЕПОДАВАТЕЛЬ	Ящун Татьяна Викторовна
СЕМЕСТР	5 семестр, 2026-2027

ПРАКТИЧЕСКАЯ РАБОТА № 3
ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ В PYTHON. КЛАССЫ, ОБЪЕКТЫ И ВЗАИМОДЕЙСТВИЕ ОБЪЕКТОВ

ЦЕЛЬ РАБОТЫ

Изучить основные принципы объектно-ориентированного программирования в Python и преобразовать программную структуру, построенную на функциях и коллекциях, в объектную модель предметной области.

СОДЕРЖАНИЕ

1. ЗАДАЧИ РАБОТЫ 2
2. ПОРЯДОК ВЫПОЛНЕНИЯ И ОФОРМЛЕНИЯ РАБОТЫ 3
3. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 4
  3.1. Объектно-ориентированный подход 4
  3.2. Класс и объект 5
  3.3. Атрибуты объекта 6
  3.4. Конструктор `__init__` 6
  3.5. Методы объекта и разделение ответственности 7
  3.6. Метод `__str__` 7
4. ПРАКТИЧЕСКАЯ ЧАСТЬ. ВЫПОЛНЕНИЕ МАТЕРИАЛОВ ЯНДЕКС ПРАКТИКУМА 8
  4.1. Объекты и классы 8
  4.2. Материалы ПР3. Знакомство с ООП 8
  4.3. Расширенные возможности Python 8
5. ИНДИВИДУАЛЬНЫЙ СКВОЗНОЙ ПРОЕКТ 9
  5.1. Подготовка рабочего проекта 9
    5.1.1. Используется проект из ПР2 10
    5.1.2. Проверка состояния проекта 10
    5.1.3. Создание контрольной точки Git 11
    5.1.4. Анализ предметной области 11
    5.1.5. Определение будущих классов 11
    5.1.6. Результат первой части практической работы 12
  5.2. Требования к доработке индивидуального проекта 12

5.3. Что изменяется по сравнению с ПР2 13
5.4. Последовательная доработка проекта 14
5.4.1. Состояние проекта после ПР2 14
5.4.2. Определение классов предметной области 14
5.4.3. Судьба функций ПР2 15
5.4.4. Класс `Room` 16
5.4.5. Класс `User` 19
5.4.6. Класс `Booking` 22
5.4.7. Взаимодействие объектов 25
5.4.8. Организация `main.py` 28
5.4.9. Сохранение объектов в JSON 31
5.4.10. Адаптация автоматизированных тестов 35
5.5. Обновление README.md 39
5.6. Проверка и фиксация результата 42
5.6.1. Проверка программы 42
5.6.2. Запуск тестов 43
5.6.3. Проверка качества кода 43
5.6.4. Проверка изменений Git 43
6. РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ 44
7. ФОРМА КОНТРОЛЯ И КРИТЕРИИ ОЦЕНКИ 45
7.1. Чек-лист выполнения материалов Яндекс Практикума 46
7.2. Чек-лист выполнения индивидуального проекта 46
7.3. Критерии оценки 47
7.4. Требования к отчету 47
8. КОНТРОЛЬНЫЕ ВОПРОСЫ 48
9. ИТОГ 50

1. ЗАДАЧИ РАБОТЫ

В ходе практической работы необходимо:

- выполнить материалы Яндекс Практикума по объектно-ориентированному программированию;
- изучить классы и объекты в Python;
- создать собственные классы;
- научиться определять атрибуты и методы объектов;
- реализовать строковое представление объектов;
- рассмотреть основные принципы ООП;
- изучить наследование, полиморфизм и инкапсуляцию;
- изучить декораторы и декораторы методов класса;
- проанализировать структуру индивидуального проекта после ПР2;
- преобразовать основные сущности проекта в объекты;
- организовать взаимодействие объектов;
- сохранить существующую функциональность проекта после перехода к объектной модели;
- обновить тесты и документацию проекта;
- зафиксировать результат в Git.

2. ПОРЯДОК ВЫПОЛНЕНИЯ И ОФОРМЛЕНИЯ РАБОТЫ

1. Войдите в личный аккаунт Яндекс с использованием персональной корпоративной почты `edu.mirea.ru` и перейдите в курс «Бэкенд-разработка».

2. Перед началом работы ознакомьтесь:

- с целью и задачами практической работы;
- с перечнем материалов Яндекс Практикума, которые необходимо выполнить;
- с материалом сквозного примера индивидуального проекта;
- с требованиями к отчету;
- с формой контроля и перечнем контрольных вопросов.

3. Выполните материалы Яндекс Практикума по объектно-ориентированному программированию и декораторам. В ПР3 выполняются темы «Объекты и классы» и «Знакомство с ООП» раздела 03, а также темы «Декораторы» и «Декораторы для методов класса». Состав выполняемых материалов приведен ниже.

№	Тема	Урок (материал)	Выполнить на ПР3
1	Объекты и классы	«Введение. ООП»	✓
2	Объекты и классы	«Классы и объекты в Python»	✓
3	Объекты и классы	«Собственные классы и объекты»	✓
4	Объекты и классы	«Атрибуты класса и объекта»	✓
5	Объекты и классы	«Методы объекта»	✓
6	Объекты и классы	«Магический метод __str__»	✓
7	Объекты и классы	«Практика по теме»	✓
8	Знакомство с ООП	«Принципы ООП»	✓
9	Знакомство с ООП	«Наследование»	✓
10	Знакомство с ООП	«Полиморфизм»	✓
11	Знакомство с ООП	«Инкапсуляция»	✓
12	Знакомство с ООП	«Практика по теме»	✓
13	Расширенные возможности Python	«Декораторы»	✓
14	Расширенные возможности Python	«Декораторы для методов класса»	✓
15	Расширенные возможности Python	«Практика по теме»	✓
16	Финальный проект спринта: «Изгиб Питона»	«Подготовка проекта к автотестам»	✓
17	Финальный проект спринта: «Изгиб Питона»	«Проверка проекта автотестами»	✓
18	Финальный проект спринта: «Изгиб Питона»	«Проект. Игра «Изгиб Питона»»	✓

Примечание. Материалы ПР1 и ПР2 повторно в ПР3 не выполняются, однако продолжают использоваться в индивидуальном проекте: коллекции, функции, модули, исключения, работа с файлами, аннотации типов, тесты pytest и проверка flake8 остаются основой проекта.

Отметьте выполненные материалы в чек-листе практической работы (подраздел 7.1).

4. Подготовьте рабочий проект к преобразованию (подраздел 5.1): откройте репозиторий индивидуального проекта из ПР2, проверьте работоспособность программы

и тестов, создайте контрольную точку в Git, проанализируйте предметную область и составьте таблицу будущих классов с атрибутами и методами.

5. Выполните преобразование проекта (раздел 5.2):

- представьте основные сущности предметной области в виде классов,
- реализуйте конструктор `__init__` и метод `__str__`,
- определите методы объектов,
- организуйте взаимодействие объектов,
- адаптируйте загрузку и сохранение JSON-файлов к объектной модели
- переработайте функции ПР2.

Существующая функциональность проекта должна быть сохранена.

6. Адаптируйте автоматизированные тесты к объектной модели (подраздел 5.3) и убедитесь, что все тесты проходят:
7. Проверьте качество кода: все обнаруженные замечания необходимо устранить.
8. Обновите файл README.md проекта (подраздел 5.5): опишите основные классы, их атрибуты и методы, взаимодействие объектов, актуальную структуру проекта, запуск программы и тестов.
9. Зафиксируйте результат в Git:

- проверьте состояние репозитория;
- просмотрите изменения;
- добавьте изменения;
- создайте информативный `commit`;
- при использовании удаленного репозитория отправьте изменения.

В репозитории должна быть сохранена версия индивидуального проекта с объектной моделью, соответствующая ПР3.

10. Подготовьте отчет о выполнении работы в соответствии с требованиями подраздела 7.4 и представьте результаты преподавателю.

3. ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Перед выполнением практической части рекомендуется ознакомиться с краткими теоретическими сведениями. Они помогают восстановить контекст: показать, что такое классы и объекты в Python, зачем они нужны и как связаны с индивидуальным проектом. Приведенные сведения не заменяют материалы Яндекс Практикума, а организуют их изучение.

3.1. Объектно-ориентированный подход

При функциональном подходе программа рассматривается как набор данных и функций, которые эти данные обрабатывают. Именно так была устроена программа

индивидуального проекта после ПР2. Например, в ПР2 помещение могло представляться словарём:

```
room = {
    "id": 1,
    "name": "Аудитория 301",
    "capacity": 30
}
```

Работа с ним выполнялась отдельными функциями: `add_room()`, `find_room()`, `check_room_capacity()`, `sort_rooms()`.

При объектно-ориентированном подходе данные и операции над ними объединяются в объект. Например, для помещения определяется класс `Room`, после создания объекта `room = Room(...)` его данные становятся атрибутами объекта, а операции над ним – методами. Соотношение понятий класса и объекта показано на рис. 1.



Рисунок 1 – Соотношение понятий класса и объекта

Класс можно рассматривать как описание того, каким образом устроены объекты определённого типа. Объект – конкретный экземпляр класса. Например, после описания класса `Room` можно создать несколько объектов этого класса с разными значениями атрибутов (рис. 2).

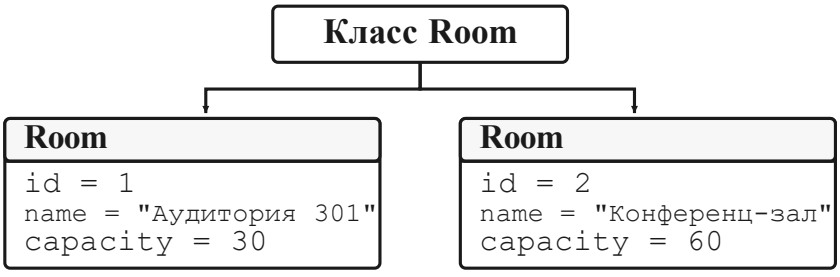


Рисунок 2 – Объекты одного класса

Оба объекта относятся к одному классу, но содержат разные значения атрибутов.

3.2. Класс и объект

Минимальный класс в Python:

```
class Room:
    pass
```

Создание объекта: `room = Room()`. Можно создать несколько объектов одного класса: `room_1 = Room()`, `room_2 = Room()`.

При этом выражение `room_1 is room_2` вернёт `False`, поскольку это два разных объекта.

3.3. Атрибуты объекта

Атрибуты содержат данные, характеризующие объект. Например, помещение может иметь идентификатор, название и вместимость. Сначала класс можно определить следующим образом:

```
class Room:
    def __init__(self, room_id, name, capacity):
        self.id = room_id
        self.name = name
        self.capacity = capacity
```

Создание объекта:

```
room = Room(
    1,
    "Аудитория 301",
    30)
```

После создания выражения `room.id`, `room.name`, `room.capacity` обращаются к атрибутам конкретного объекта.

Важно различать `room.id` и `Room`: `Room` – класс, `room` – объект класса `Room`.

3.4. Конструктор `__init__`

Метод `__init__` используется для начальной настройки объекта при его создании. Общий вид:

```
class ClassName:
    def __init__(self, parameter):
        self.attribute = parameter
```

При выполнении `object = ClassName(value)` переданное значение сохраняется в атрибуте объекта.

В методах класса параметр `self` обозначает текущий объект. Например:

```
class Room:
    def __init__(self, room_id, name, capacity):
        self.id = room_id
        self.name = name
        self.capacity = capacity
```

Здесь `self.id`, `self.name`, `self.capacity` относятся к конкретному объекту.

3.5. Методы объекта и разделение ответственности

Метод – функция, определённая внутри класса. Например:

```
class Room:
    def is_suitable_for(self, people_count):
        """Проверить, подходит ли помещение по вместимости."""
        pass
```

Вызов метода: `room.is_suitable_for(20)`. Метод получает доступ к атрибутам объекта через `self`:

```
class Room:
    def is_suitable_for(self, people_count):
        return self.capacity >= people_count
```

Теперь `room.is_suitable_for(20)` проверяет вместимость именно объекта `room`.

При переходе от функций к объектам не требуется механически превращать каждую функцию в метод. Необходимо определить, какой объект отвечает за конкретное действие. Состав класса `Room` показан на рис.3.

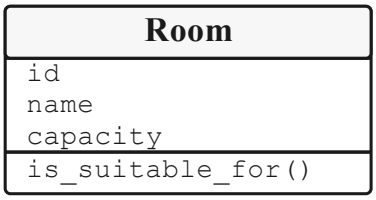


Рисунок 3 - Атрибуты и методы класса Room

Проверка вместимости непосредственно относится к помещению, поэтому её естественно реализовать методом `Room`. В то же время загрузка данных из JSON-файла не является поведением конкретного помещения: функции `load_rooms()` и `save_rooms()` могут оставаться отдельными функциями модуля хранения.

Примечание. Метод должен описывать поведение объекта, а не просто переносить существующую функцию внутрь класса.

3.6. Метод `__str__`

Для объектов часто требуется определить удобное текстовое представление. Для этого используется специальный метод `__str__`. Например:

```
class Room:
    def __str__(self):
        return f"{self.name}, {self.capacity} мест"
```

После этого `print(room)` может вывести: `Аудитория 301, 30 мест`.

Это особенно удобно для основного интерфейса приложения. Вместо формирования строки в `main.py`: `print(room["name"], room["capacity"])` – можно

использовать `print(room)`. Таким образом, объект самостоятельно определяет, как представить себя пользователю.

4. ПРАКТИЧЕСКАЯ ЧАСТЬ. ВЫПОЛНЕНИЕ МАТЕРИАЛОВ ЯНДЕКС ПРАКТИКУМА

Практическая работа выполняется с использованием курса «Бэкенд-разработчик» Яндекс Практикума и продолжает работу с материалами, начатую в ПР1 и ПР2. Материалы курса выступают основным учебным ресурсом: теория, примеры и практические задания каждого раздела выполняются последовательно.

Примечание. Материалы Яндекс Практикума используются как основной учебный ресурс. Дополнительные материалы настоящих методических указаний применяются для закрепления: теоретические сведения приведены в разделе 4, развитие индивидуального проекта – в разделе 6.

4.1. Объекты и классы

В рамках блока «Объекты и классы» выполняются темы:

- Урок «Введение. ООП»
- Урок «Классы и объекты в Python»
- Урок «Собственные классы и объекты»
- Урок «Атрибуты класса и объекта»
- Урок «Методы объекта»
- Урок «Магический метод `__str__`
- Урок «Практика по теме».

4.2. Знакомство с ООП

В рамках блока «Знакомство с ООП» выполняются темы:

- Урок «Принципы ООП»
- Урок «Наследование»
- Урок «Полиморфизм»
- Урок «Инкапсуляция»
- Урок «Практика по теме»

4.3. Расширенные возможности Python

В ПР3 впервые выполняются материалы о декораторах.

- Урок «Декораторы»
- Урок «Декораторы для методов класса»
- Урок «Практика по теме»

4.4. Финальный проект спринта: «Изгиб Питона»

В рамках темы «Финальный проект спринта: «Изгиб Питона»» выполняются все пункты итогового практического задания.

5. ИНДИВИДУАЛЬНЫЙ СКВОЗНОЙ ПРОЕКТ

В рамках ПР3 продолжается разработка индивидуального проекта, начатого в ПР1 и доработанного в ПР2. Существующий проект не создается заново: функциональная модель, построенная на коллекциях и функциях, перерабатывается в объектно-ориентированную модель. В ПР2 данные предметной области представлялись словарями и списками, а работа с ними выполнялась отдельными функциями. В ПР3 основные сущности предметной области становятся классами, а данные – объектами этих классов.

Таким образом, реализуется принцип развития проекта:

- ПР1 – проверка одного сценария бронирования;
- ПР2 – создание функциональной программы, работающей с множеством помещений и бронирований;
- ПР3 – преобразование предметной области в систему объектов и классов.

Принцип развития проекта показан на рис. 4.

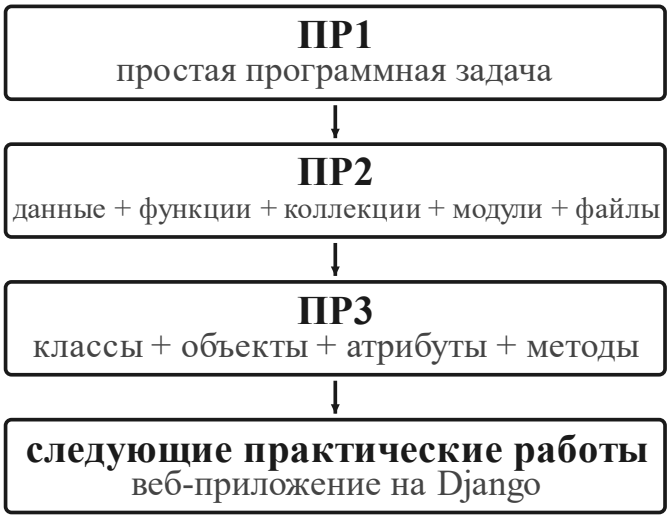


Рисунок 4 - Принцип развития индивидуального проекта

Подраздел 5.1 описывает действия по подготовке проекта к доработке, подраздел 5.2 определяет обязательные требования к доработке проекта, подраздел 5.3 показывает, что изменяется по сравнению с ПР2, подраздел 5.4 содержит последовательный пример переработки проекта, подраздел 5.5 описывает проверку и фиксацию результата.

5.1. Подготовка рабочего проекта

Первая часть практической работы выполняется до преобразования проекта. На этом этапе проверяется работоспособность проекта из ПР2, создается контрольная точка в Git, анализируется предметная область и определяются будущие классы. Пункты 5.1.1–5.1.5 выполняются последовательно; их результат контролируется в пункте 5.1.7 перед переходом к подразделу 5.2.

5.1.1. Используется проект из ПР2

Перед началом работы необходимо открыть репозиторий индивидуального проекта, созданный в ПР1 и доработанный в ПР2. Создавать отдельный репозиторий для ПР3 не требуется.

Структура проекта на начало ПР3 должна соответствовать результату предыдущей практической работы. Для примера:

```
room_booking/
│
├── README.md
├── requirements.txt
├── main.py
├── storage.py
├── utils.py
│
├── models/
│   ├── __init__.py
│   ├── rooms.py
│   ├── bookings.py
│   └── users.py
│
├── data/
│   ├── rooms.json
│   ├── bookings.json
│   └── users.json
│
└── tests/
    ├── test_rooms.py
    ├── test_bookings.py
    └── test_users.py
```

Для собственного проекта названия модулей и файлов могут отличаться, если они соответствуют предметной области.

5.1.2. Проверка состояния проекта

Перед внесением изменений необходимо убедиться, что проект из ПР2 запускается. Проверить:

1. Запуск программы
2. Основные операции приложения
3. Загрузку данных из файлов
4. Сохранение данных
5. Обработку ошибочного ввода
6. Выполнение автоматических тестов
7. Проверку качества кода

Если после ПР2 в проекте имеются ошибки, сначала необходимо определить их причину и зафиксировать исходное состояние.

5.1.3. Создание контрольной точки Git

Перед преобразованием программы к объектной модели рекомендуется сохранить состояние проекта в Git:

```
git status
git add .
git commit -m "Подготовка проекта к переходу на ООП"
```

После этого дальнейшие изменения можно сравнивать с рабочей версией ПР2: при необходимости всегда доступно исходное состояние проекта, а каждый шаг преобразования фиксируется отдельным коммитом.

5.1.4. Анализ предметной области

Перед созданием классов необходимо определить основные сущности собственного проекта. Для системы бронирования помещений это:

Сущность	Назначение
`Room`	помещение, доступное для бронирования
`User`	пользователь системы
`Booking`	бронирование помещения пользователем

Для другого проекта набор сущностей будет другим. Например:

- интернет-магазин: `Product`, `Customer`, `Order`;
- система управления задачами: `User`, `Project`, `Task`, `Comment`;
- библиотека: `Book`, `Reader`, `Loan`.

Сущность становится кандидатом на отдельный класс, если у неё есть:

- собственные данные;
- собственное состояние;
- операции, которые логически относятся к этой сущности;
- необходимость создавать несколько экземпляров такой сущности.

5.1.5. Определение будущих классов

Для каждой сущности необходимо определить, какие данные она хранит и какое поведение имеет:

Сущность	Атрибуты	Основное поведение
`Room`	`id, name, capacity`	проверка вместимости, строковое представление
`User`	`id, name, email`	создание пользователя, строковое представление
`Booking`	`id, room, booking_date, user, is_cancelled`	создание, отмена, определение статуса, строковое представление

Для собственного проекта необходимо составить аналогичную таблицу.

При этом не следует переносить в класс все функции модуля автоматически. Например, модуль `rooms.py` может содержать класс `Room` и отдельные функции обработки коллекции помещений. А модуль `storage.py` может продолжать содержать функции `load_rooms()` и `save_rooms()`, поскольку их задача – работа с внешним хранилищем, а не поведение отдельного объекта `Room`.

5.1.6. Результат первой части практической работы

К началу выполнения пункта 6 должен быть подготовлен:

- существующий проект из ПР2;
- рабочая версия проекта;
- сохранённая контрольная точка в Git;
- определённые основные сущности предметной области;
- предварительный список классов;
- перечень атрибутов будущих объектов;
- перечень операций, которые целесообразно реализовать как методы;
- понимание того, какие функции ПР2 останутся обычными функциями;
- выполненные материалы Яндекс Практикума по ООП и декораторам.

Дальнейшая работа выполняется непосредственно на основе проекта из ПР2 и начинается с построения его объектной модели.

5.2. Требования к доработке индивидуального проекта

В существующем проекте необходимо:

- определить основные сущности предметной области;
- представить основные сущности в виде классов;
- создать объекты этих классов;
- определить атрибуты объектов;
- определить методы, описывающие поведение объектов;
- реализовать конструктор `__init__`;
- реализовать строковое представление объектов с помощью `__str__`;
- организовать коллекции объектов классов место коллекций словарей;
- организовать взаимодействие объектов;
- заменить или адаптировать коллекции словарей из ПР2 на коллекции объектов;
- перенести в методы операции, относящиеся к конкретному объекту; функции обработки коллекций сохранить как обычные функции;
- сохранить существующую функциональность проекта;
- сохранить работу с JSON-файлами;
- адаптировать загрузку данных из JSON к созданию объектов;
- адаптировать сохранение объектов в JSON;

- сохранить обработку исключений;
- адаптировать существующие автоматизированные тесты;
- сохранить проверку кода с помощью flake8;
- обновить аннотации типов;
- обновить `README.md`;
- зафиксировать результат в Git.

Не следует механически превращать каждую функцию ПР2 в метод класса. Функция становится методом в том случае, если выполняемая операция относится к конкретному объекту или его состоянию.

Функции общего назначения, например `input_int()`, `input_date()`, а также операции сохранения и загрузки данных, могут оставаться обычными функциями.

5.3. Что изменяется по сравнению с ПР2

В ПР2 проект имел функциональную структуру: данные представлялись словарями и списками, над ними работали функции обработки, результат возвращался пользователю (рис. 5).

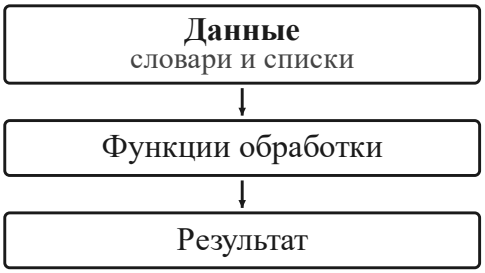


Рисунок 5 - Функциональная модель проекта после ПР2

Например, помещение представлялось словарём:

```
room = {
    "id": 1,
    "name": "Аудитория 301",
    "capacity": 30
}
```

Проверка вместимости выполнялась отдельной функцией: `check_room_capacity(rooms, room_id, min_capacity)`.

В ПР3 помещение становится объектом:

```
room = Room(
    room_id=1,
    name="Аудитория 301",
    capacity=30
)
```

А операция, относящаяся непосредственно к помещению, становится методом: `room.is_suitable_for(20)`. Таким образом, данные и связанное с ними поведение объединяются в одном объекте.

В ПР2 использовалось обращение `rooms → check_room_capacity()`; в ПР3 те же данные и операции образуют состав класса `Room`: атрибуты `id`, `name`, `capacity` и метод `is_suitable_for()`.

При этом проект не переписывается с нуля. Существующие функции, модули, JSON-файлы, тесты и интерфейс используются как основа для переработки.

5.4. Последовательная доработка проекта

Подраздел показывает переработку сквозного примера – проекта «Система бронирования помещений» – в виде последовательности шагов: от анализа состояния проекта до обновления `README.md`. Для собственного проекта выполняются те же шаги с сущностями своей предметной области.

5.4.1. Состояние проекта после ПР2

После ПР2 в проекте уже имеются:

- несколько помещений;
- несколько бронирований;
- функции работы с помещениями;
- функции работы с бронированиями;
- модули;
- JSON-файлы;
- обработка исключений;
- консольный интерфейс;
- автоматизированные тесты;
- README.md;
- Git-репозиторий.

В ПР3 эта структура сохраняется, но способ представления предметной области изменяется.

5.4.2. Определение классов предметной области

Для проекта «Система бронирования помещений» выделяются три основные сущности: `Room` – помещение; `User` – пользователь; `Booking` – бронирование. Связь между объектами показана на рис. 8.

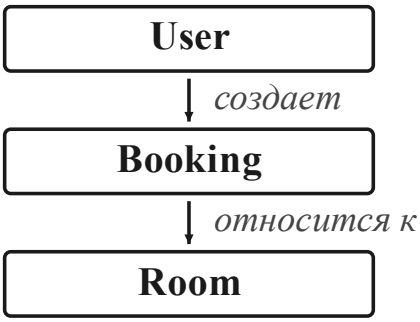


Рисунок 6 - Взаимодействие объектов предметной области

Бронирование должно хранить ссылки на объекты `User` и `Room`, а не только их идентификаторы. Например:

```
booking = Booking(
    booking_id=1,
    room=room,
    booking_date=booking_date,
    user=user
)
```

Такой подход позволяет объектам взаимодействовать друг с другом.

5.4.3. Судьба функций ПР2

При переработке проекта необходимо определить, какие функции сохраняются, какие изменяются, а какие операции переносятся в методы классов.

Модуль ПР2	Функция ПР2	Что происходит в ПР3
`main.py`	`main()`	сохраняется, адаптируется для работы с объектами `Room` и `Booking`
`models/rooms.py`	`show_rooms()`	сохраняется, адаптируется для вывода объектов `Room`
`models/bookings.py`	`show_bookings()`	сохраняется, адаптируется для вывода объектов `Booking`
`models/rooms.py`	`add_room()`	изменяется: создаёт объект Room, добавляет его в коллекцию и возвращает созданный объект
`models/rooms.py`	`find_room()`	сохраняется по назначению, адаптируется для работы с объектами `Room`
`models/rooms.py`	`check_room_capacity()`	перерабатывается: проверка возможности размещения переносится в метод `Room.is_suitable_for()`
`models/rooms.py`	`filter_rooms_by_capacity()`	изменяется для работы с коллекцией объектов `Room`
`models/rooms.py`	`sort_rooms()`	сохраняется по назначению, адаптируется для сортировки объектов `Room`
`models/bookings.py`	`is_room_available()`	изменяется для работы с объектами `Room` и `Booking`
`models/bookings.py`	`create_booking()`	изменяется: создаёт и возвращает объект `Booking`

models/bookings.py	cancel_booking()	перерабатывается с учётом состояния объекта Booking: изменяет его состояние, не удаляя объект из коллекции
models/bookings.py	get_booking_status()	перерабатывается с учётом состояния объекта Booking
storage.py	load_rooms()	сохраняется по назначению: преобразует данные JSON в объекты Room
storage.py	save_rooms()	сохраняется по назначению: преобразует объекты Room в данные JSON
storage.py	load_bookings()	сохраняется по назначению: преобразует данные JSON в объекты Booking
storage.py	save_bookings()	сохраняется по назначению: преобразует объекты Booking в данные JSON
utils.py	input_int()	остаётся обычной вспомогательной функцией
utils.py	input_date()	остаётся обычной вспомогательной функцией

Главное изменение ПР3 заключается не в количестве методов, а в распределении ответственности между объектами.

5.4.4. Класс `Room`

Поскольку классы `Room`, `User` и `Booking` размещаются в отдельных файлах пакета `models`, файл `models/__init__.py` используется для их импорта из пакета. В него необходимо добавить:

```
from .rooms import Room
from .users import User
from .bookings import Booking
```

После этого классы можно импортировать из пакета `models`:

```
from models import Room, User, Booking
```

Относительная точка в импортах (`.rooms`, `.users`, `.bookings`) указывает на модули, расположенные внутри пакета `models`.

Класс `Room` описывает помещение. Основные атрибуты:

- `id` – идентификатор;
- `name` – название;
- `capacity` – вместимость.

Каркас класса:

```
class Room:

    """Помещение для бронирования."""

    def __init__(
        self,
        room_id: int,
        name: str,
        capacity: int,
```

```
    ) -> None:
        """Создать объект помещения.
        TODO:
        сохранить переданные данные
        в атрибутах объекта.
        """
        pass

    def is_suitable_for(
        self,
        people_count: int,
    ) -> bool:
        """Проверить вместимость помещения.
        TODO:
        вернуть True, если помещение подходит
        для указанного количества людей.
        """
        pass

    def __str__(self) -> str:
        """Вернуть строковое представление помещения.
        TODO:
        сформировать удобное описание помещения.
        """
        pass
```

При реализации класса необходимо:

- сохранить переданные значения в атрибутах объекта;
- использовать аннотации типов для параметров и возвращаемых значений;
- реализовать конструктор `__init__()`;
- реализовать `is_suitable_for()` как метод экземпляра, использующий данные конкретного объекта;
- реализовать `__str__()` для получения удобного строкового представления объекта;
- сохранить назначение методов, реализованных на предыдущем этапе разработки.

После реализации с объектом можно работать следующим образом:

```
room = Room(1, "Аудитория 301", 30)
print(room.name)
print(room.capacity)
print(room.is_suitable_for(20))
print(room)
```

После создания класса `Room` необходимо переработать функции модуля `rooms.py`, ранее работавшие с данными помещений.

При переработке функций:

- заменить работу со словарями помещений на работу с объектами `Room`;
- изменить тип параметров и возвращаемых значений функций в соответствии с новой структурой данных;
- передавать в функции коллекцию объектов `Room`;
- обращаться к данным помещения через атрибуты объекта, например `room.name`, `room.capacity`, `room.id`;
- использовать методы объекта `Room`, если соответствующая операция относится непосредственно к помещению;
- сохранить назначение и результаты работы существующих функций;
- не дублировать в функциях логику, которая уже реализована в методах класса `Room`.

Необходимо переработать следующие функции:

- `add_room()` – создание объекта `Room` и добавление его в коллекцию;
- `find_room()` – поиск объектов `Room` по названию;
- `check_room_capacity()` – поиск помещения и проверка его вместимости;
- `filter_rooms_by_capacity()` – отбор объектов `Room` по вместимости;
- `sort_rooms()` – сортировка коллекции объектов `Room`;
- `show_rooms()` – вывод информации об объектах `Room`.

Например, проверка вместимости должна использовать метод объекта:

```
room.is_suitable_for(min_capacity)
```

а не дублировать его логику непосредственно в функции.

В результате функции модуля `rooms.py` должны работать с коллекцией объектов `List[Room]`, а не с коллекцией словарей.

Для всех переработанных функций необходимо указать аннотации типов параметров и возвращаемого значения.

Например:

```
def find_room(
    rooms: List[Room],
    query: str,
) -> List[Room]:
    ...
def check_room_capacity(
    rooms: List[Room],
    room_id: int,
    min_capacity: int,
) -> bool:
    ...
```

Тип возвращаемого значения должен соответствовать фактическому результату работы функции. Если функция изменяет переданную коллекцию и не возвращает результат, используется `-> None`.

Декоратор `@staticmethod` используется для методов, которые логически относятся к классу, но не используют данные конкретного объекта и не требуют параметра `self`. Например:

```
class Room:

    @staticmethod
    def validate_capacity(
        capacity: int,
    ) -> bool:
        """Проверить корректность вместимости."""
        ...
```

Такой метод вызывается непосредственно через класс:

```
Room.validate_capacity(30)
```

5.4.5. Класс `User`

Класс `User` описывает пользователя системы. Основные атрибуты:

- `id`;
- `name`;
- `email`.

Каркас класса:

```
class User:

    """Пользователь системы."""

    def __init__(
        self,
        user_id: int,
        name: str,
        email: str,
    ) -> None:
        """Создать объект пользователя.
        TODO:
        сохранить переданные данные
        в атрибутах объекта.
        """
        pass

    def __str__(self) -> str:
        """Вернуть строковое представление пользователя.
```

```
        TODO:
        сформировать удобное описание пользователя.
        """
        pass
```

При реализации класса необходимо:

- сохранить переданные значения в атрибутах объекта;
- использовать аннотации типов для параметров и возвращаемых значений;
- реализовать конструктор `__init__()`;
- реализовать `__str__()` для получения строкового представления объекта;
- сохранить назначение методов, реализованных на предыдущем этапе разработки.

После реализации с объектом можно работать следующим образом:

```
user = User(
    1,
    "Иван Петров",
    "ivan@example.com",
)

print(user.name)
print(user.email)
print(user)
```

При загрузке пользователей из JSON необходимо преобразовать структуру данных в объект `User`. Для этого используется метод класса `from_data()`, создаваемый с помощью декоратора `@classmethod`:

```
class User:

    @classmethod
    def from_data(
        cls,
        data: dict,
    ) -> "User":
        """Создать пользователя из набора данных.

        TODO:
        получить значения из data
        и вернуть новый объект User.
        """
        pass
```

Метод должен получить из `data` значения `id`, `name` и `email` и создать на их основе объект `User`. Вызов метода:

```
user = User.from_data(data)
```

Использование `@classmethod` в данном случае позволяет создавать объект через сам класс `User`. Параметр `cls` содержит ссылку на класс, для которого вызывается метод.

После создания класса User необходимо переработать функции модуля `users.py`, ранее работавшие с данными пользователей.

При переработке функций:

- заменить работу со словарями пользователей на работу с объектами `User`;
- изменить тип параметров и возвращаемых значений функций в соответствии с новой структурой данных;
- передавать в функции коллекцию объектов `User`;
- обращаться к данным пользователя через атрибуты объекта, например user.name, `user.email`, `user.id`;
- использовать методы класса `User`, если соответствующая операция реализована в классе;
- сохранить назначение и результаты работы существующих функций.

Необходимо переработать следующие функции:

- `add_user()` – создание объекта User и добавление его в коллекцию;
- `find_user()` – поиск объектов User по имени или адресу электронной почты;
- `show_users()` – вывод информации об объектах User.

Функция `add_user()` должна создавать объект `User` и добавлять его в коллекцию, а не формировать словарь с данными пользователя. Результатом переработки должна быть коллекция `List[User]`, а не коллекция словарей.

Для всех методов класса и переработанных функций необходимо указать аннотации типов параметров и возвращаемого значения.

В частности:

- `__init__() - -> None;`
- `__str__() - -> str;`
- `from_data() - -> User;`
- `add_user() - -> None;`
- `find_user() - -> List[User];`
- `show_users() - -> None.`

Тип возвращаемого значения должен соответствовать фактическому результату работы метода или функции.

После переработки модуль `users.py` должен работать с объектами `User` на всех этапах: создание, поиск, вывод и восстановление объектов из данных JSON.

5.4.6. Класс `Booking`

Класс `Booking` описывает бронирование помещения пользователем. Основные атрибуты:

- `id` – идентификатор бронирования;
- `room` – объект `Room`, связанный с бронированием;
- `booking_date` – дата бронирования;
- `user` – объект `User`, связанный с бронированием;
- `is_cancelled` – признак отмены бронирования.

Каркас класса:

```
from .rooms import Room
from .users import User

class Booking:

    """Бронирование помещения."""

    def __init__(
        self,
        booking_id: int,
        room: Room,
        booking_date: str,
        user: User,
    ) -> None:
        """Создать объект бронирования.
        TODO:
        сохранить идентификатор,
        помещение, дату и пользователя.
        """
        pass

    def cancel(self) -> None:
        """Отменить бронирование.
        TODO:
        изменить состояние бронирования.
        """
        pass

    def __str__(self) -> str:
        """Вернуть строковое представление бронирования.
        TODO:
        сформировать описание бронирования
        с учетом его состояния.
        """
```

```
        pass
```

При реализации класса необходимо:

- сохранить переданные значения в атрибутах объекта;
- использовать аннотации типов;
- хранить в атрибуте `room` объект `Room`, а не идентификатор помещения;
- хранить в атрибуте `user` объект `User`, а не идентификатор пользователя;
- реализовать метод `cancel()`, изменяющий состояние бронирования;
- реализовать `__str__()` для получения удобного строкового представления объекта;
- предусмотреть отображение состояния отменённого и активного бронирования.

После создания объектов `Room` и `User` бронирование может быть создано следующим образом:

```
booking = Booking(
    1,
    room,
    "2026-09-15",
    user,
)

print(booking.room.name)
print(booking.user.name)
print(booking.booking_date)
print(booking)
```

Отмена бронирования выполняется через метод объекта:

```
booking.cancel()
```

При отмене объект `Booking` не удаляется из коллекции. Его состояние изменяется с помощью атрибута `is_cancelled`.

`Booking` связывает объекты `Room` и `User`. Вместо хранения только идентификаторов `room_id`, `user_id` объект `Booking` должен содержать ссылки на соответствующие объекты `room: Room`, `user: User`

Благодаря этому данные связанных объектов доступны непосредственно через объект бронирования:

```
booking.room.name
booking.room.capacity
booking.user.name
booking.user.email
```

После создания класса `Booking` необходимо переработать функции модуля `bookings.py`, ранее работавшие с данными бронирований.

При переработке функций:

- заменить работу со словарями бронирований на работу с объектами `Booking`;
- использовать коллекцию объектов `Booking`;
- изменить типы параметров и возвращаемых значений;
- передавать в функции объекты `Room` и `User`, когда они необходимы для создания или проверки бронирования;
- использовать атрибуты и методы объектов `Booking`, `Room` и `User`;
- сохранить существующие правила работы приложения.

Необходимо переработать следующие функции:

- `is_room_available()` – проверка доступности помещения с учетом активных бронирований;
- `create_booking()` – создание объекта `Booking` и добавление его в коллекцию;
- `cancel_booking()` – поиск бронирования и вызов его метода `cancel()`;
- `get_booking_status()` – формирование сообщения о доступности помещения;
- `show_bookings()` – вывод информации об объектах `Booking`.

При создании бронирования необходимо учитывать, что активное бронирование блокирует помещение на соответствующую дату.

Отменённое бронирование не должно блокировать помещение:

```
активное бронирование
        ↓
помещение недоступно

отменённое бронирование
        ↓
помещение доступно
```

При отмене бронирование не удаляется из списка. Это позволяет сохранить информацию о ранее созданном бронировании.

Для всех методов класса и переработанных функций необходимо указать аннотации типов параметров и возвращаемых значений. В частности:

- `__init__() - -> None;`
- `cancel() - -> None;`
- `__str__() - -> str;`
- `is_room_available() - -> bool;`
- `create_booking() - -> Optional[Booking];`
- `cancel_booking() - -> bool;`
- `get_booking_status() - -> str;`

- `show_bookings() - -> None.`

Результат `create_booking()` может отсутствовать, если помещение уже занято. Поэтому для этой функции используется тип `Optional[Booking]`.

Состояние бронирования хранится в атрибуте `is_cancelled`. В рамках данной работы отдельное свойство `status` создавать не требуется. Формирование текстового сообщения о доступности помещения выполняется функцией `get_booking_status()`.

Метод `cancel()` отвечает за изменение состояния объекта `Booking`, а функции работы с коллекцией бронирований – за поиск объектов, проверку доступности и организацию взаимодействия между бронированиями, помещениями и пользователями.

В результате модуль `bookings.py` должен работать с объектами `Booking`, связанными с объектами `Room` и `User`, при сохранении существующих правил бронирования и отмены.

5.4.7. Взаимодействие объектов

После создания классов необходимо организовать их взаимодействие в соответствии с предметной областью.

Объект `Booking` связывает объект помещения `Room` с объектом пользователя `User`. Например:

```
user = User(
    user_id=1,
    name="Иван Иванов",
    email="ivan@example.com",
)

room = Room(
    room_id=1,
    name="Аудитория 301",
    capacity=30,
)

booking = Booking(
    booking_id=1,
    room=room,
    booking_date="2026-09-15",
    user=user,
)
```

В результате объект `Booking` содержит ссылки на объекты предметной области:

```
booking
  ├── room → Room
```

```
└── user → User
```

Данные связанных объектов должны быть доступны через соответствующие атрибуты:

```
booking.room.name
booking.room.capacity
booking.user.name
booking.user.email
```

Необходимо обеспечить хранение объектов предметной области в коллекциях:

```
rooms = [room1, room2, room3]
users = [user1, user2]
bookings = [booking1, booking2]
```

Таким образом, после переработки проекта используются следующие типы коллекций:

- `List[Room]`
- `List[User]`
- `List[Booking]`

Функции модулей `rooms.py`, `users.py` и `bookings.py` должны работать с объектами соответствующих классов.

При переработке необходимо:

- передавать в функции коллекции объектов;
- выполнять поиск объектов по их атрибутам;
- использовать методы объектов для операций, относящихся непосредственно к объекту;
- сохранять существующую функциональность приложения;
- не преобразовывать объекты обратно в словари для выполнения обычных операций приложения;
- использовать аннотации типов для коллекций и объектов.

Например, функция проверки доступности помещения должна получать объект `Room` и коллекцию объектов `Booking`:

```
is_room_available(
    bookings,
    room,
    booking_date,
)
```

При проверке необходимо обращаться к связанному объекту помещения `booking.room.id`, а состояние бронирования проверять через `booking.is_cancelled`.

При создании бронирования функция должна сформировать объект `Booking`, связав его с существующими объектами `Room` и `User`. При отмене бронирования объект `Booking` не удаляется из коллекции. Вместо этого вызывается его метод `booking.cancel()`. После отмены объект остаётся в коллекции, но перестаёт блокировать помещение на соответствующую дату.

Объектная модель используется для работы приложения, однако JSON-файлы сохраняют идентификаторы связанных объектов.

Поэтому при сохранении бронирования:

```
Booking
    │
    ├── room → Room
    └── user → User
```

преобразуется в структуру:

```
{
    "id": ...,
    "room_id": ...,
    "booking_date": ...,
    "user_id": ...,
    "is_cancelled": ...
}
```

При загрузке выполняется обратное преобразование:

```
JSON
  ↓
room_id / user_id
  ↓
поиск объектов Room и User
  ↓
создание Booking
  ↓
Booking.room → Room
Booking.user → User
```

Таким образом, JSON используется для хранения данных, а объектная модель – для работы приложения.

Принципиальное изменение по сравнению с ПР2 заключается в переходе от работы со словарями к объектной модели.

В ПР2 использовались структуры типа `list[dict]`, `dict[int, dict]`. В ПР3 используются `List[Room]`, `List[User]`, `List[Booking]`.

Функции приложения должны работать непосредственно с объектами предметной области, а преобразование объектов в структуры JSON и обратно выполняется в модуле storage.py.

В результате классы `Room`, `User` и `Booking` становятся основными элементами модели данных приложения, а функции модулей организуют операции над коллекциями этих объектов.

5.4.8. Организация `main.py`

Основная программа должна обеспечивать взаимодействие объектов предметной области и сохранять пользовательские сценарии, реализованные на предыдущем этапе. После перехода к объектной модели `main.py` не должен самостоятельно хранить данные помещений, пользователей и бронирований в виде словарей. Основные данные передаются в виде коллекций объектов:

```
List[Room]
List[User]
List[Booking]
```

Импорт необходимых классов выполняется из пакета `models`:

```
from models import Booking, Room, User
```

Функции работы с помещениями, пользователями и бронированиями импортируются из соответствующих модулей:

```
from models.rooms import (
    add_room,
    check_room_capacity,
    find_room,
    show_rooms,
    sort_rooms,
)

from models.users import (
    add_user,
    find_user,
    show_users,
)

from models.bookings import (
    cancel_booking,
    create_booking,
    get_booking_status,
    is_room_available,
    show_bookings,
)
```

Также необходимо импортировать функции работы с коллекциями, функции загрузки и сохранения данных и вспомогательные функции ввода.

Для пользовательского сценария создания бронирования используется отдельная функция `create_new_booking()`.

Функция должна:

- получить идентификатор помещения;
- найти соответствующий объект `Room`;
- получить идентификатор пользователя;
- найти соответствующий объект `User`;
- получить дату бронирования;
- проверить возможность создания бронирования;
- вызвать функцию `create_booking()`, создать объект `Booking`, связав его с найденными объектами `Room` и `Use`;
- обработать результат операции;
- вывести сообщение пользователю.

При поиске помещения и пользователя необходимо использовать объекты соответствующих классов. Если помещение или пользователь не найдены, создание бронирования не выполняется. При успешном создании `create_booking()` возвращает объект `Booking`. Полученный объект может использоваться для вывода идентификатора созданного бронирования. Если помещение уже занято на указанную дату, `create_booking()` возвращает `None`. В этом случае необходимо вывести соответствующее сообщение и не добавлять новое бронирование в коллекцию.

Последовательность взаимодействия объектов должна соответствовать схеме:

```
Ввод ID помещения
        ↓
   поиск Room
        ↓
Ввод ID пользователя
        ↓
   поиск User
        ↓
  Ввод даты
        ↓
Проверка доступности
        ↓
Создание Booking
         │
         ├── room → Room
         └── user → User
```

При этом объект `Booking` не должен создаваться непосредственно в `main.py`. Создание бронирования выполняется функцией `create_booking()`, которая отвечает также за проверку доступности помещения и добавление нового объекта в коллекцию.

Функция `main()` должна:

- загрузить помещения из `rooms.json`;
- загрузить пользователей из `users.json`;
- загрузить бронирования из `bookings.json` с восстановлением связей с объектами `Room` и `User`;
- запустить главное меню приложения;
- передавать коллекции объектов в соответствующие функции;
- организовать выполнение пользовательских операций;
- сохранить изменённые данные перед завершением работы приложения.

При запуске приложения должны быть сформированы коллекции:

```
rooms: List[Room]
users: List[User]
bookings: List[Booking]
```

После загрузки бронирований каждый объект `Booking` должен содержать ссылки на соответствующие объекты `Room` и `User`.

Основная программа должна использовать функции модулей `rooms.py`, `users.py`, `bookings.py` и `storage.py`, не дублируя реализованную в них логику.

Перед завершением работы приложения необходимо сохранить изменённые коллекции:

- помещения – в `data/rooms.json`;
- пользователей – в `data/users.json`;
- бронирования – в `data/bookings.json`.

При сохранении бронирований объектные ссылки:

```
booking.room
booking.user
```

должны преобразовываться в идентификаторы:

```
room_id
user_id
```

Это преобразование выполняется функциями модуля `storage.py`, а не в `main.py`.

Для функций `main.py`, создаваемых или перерабатываемых в рамках ПР3, необходимо указать аннотации типов параметров и возвращаемых значений. В частности, пользовательские функции, изменяющие состояние приложения, должны иметь явно указанный тип `-> None`, если они не возвращают результат операции.

Например:

```
def create_new_booking(
    bookings: List[Booking],
```

```
    rooms: List[Room],
    users: List[User],
) -> None:
    ...
```

Функция `main()` также не возвращает результат:

```
def main() -> None:
    ...
```

После завершения переработки `main.py` должен:

- работать с объектами `Room`, `User` и `Booking`;
- передавать коллекции объектов в функции соответствующих модулей;
- не содержать коллекций словарей предметной области;
- не дублировать логику создания, поиска, проверки и отмены бронирований;
- сохранять возможность выполнения основных пользовательских сценариев;
- загружать и сохранять данные через функции `storage.py`;
- использовать аннотации типов.

Точка запуска приложения сохраняется:

```
if __name__ == "__main__":
    main()
```

Логика создания бронирования в объектной модели представлена на рисунке 7.



Рисунок 7 – Логика создания бронирования в объектной модели

5.4.9. Сохранение объектов в JSON

Использование классов не отменяет работу с JSON из ПР2. JSON остается форматом хранения данных. Например, помещение в файле по-прежнему может иметь вид:

```
{
    "id": 1,
    "name": "Аудитория 301",
    "capacity": 30
```

```
}
```

После перехода от словарей к объектам формат хранения данных в JSON сохраняется. Изменяется способ загрузки и сохранения: данные из JSON преобразуются в объекты классов `Room`, `User` и `Booking`, а перед сохранением объекты преобразуются обратно в обычные данные JSON.

Аналогично выполняется загрузка и сохранение `User` и `Booking`.

JSON-файлы не должны содержать сведения о внутренней структуре Python-объектов. В них хранятся только необходимые данные, позволяющие восстановить объекты при следующем запуске программы.

Файл `rooms.json`. Для помещения сохраняются:

- `id` – идентификатор;
- `name` – название;
- `capacity` – вместимость.

Пример:

```
[
    {
        "id": 1,
        "name": "Аудитория 301",
        "capacity": 30
    },
    {
        "id": 2,
        "name": "Конференц-зал",
        "capacity": 60
    }
]
```

При загрузке каждый элемент JSON преобразуется в объект `Room`:

```
room = Room(
    room_id=data["id"],
    name=data["name"],
    capacity=data["capacity"],
)
В результате формируется коллекция объектов:
rooms = [
    Room(...),
    Room(...),
]
```

При сохранении объект `Room` преобразуется обратно в словарь:

```
{
```

```
    "id": room.id,
    "name": room.name,
    "capacity": room.capacity,
}
```

Файл `users.json`

Для пользователя сохраняются:

- `id` – идентификатор;
- `name` – имя;
- `email` – адрес электронной почты.

Пример:

```
[
    {
        "id": 1,
        "name": "Иван Петров",
        "email": "ivan@example.com"
    },
    {
        "id": 2,
        "name": "Анна Смирнова",
        "email": "anna@example.com"
    }
]
```

При загрузке данные преобразуются в объекты User:

```
user = User(
    user_id=data["id"],
    name=data["name"],
    email=data["email"],
)
```

Файл `bookings.json`

Для бронирования сохраняются:

- `id` – идентификатор бронирования;
- `room_id` – идентификатор помещения;
- `booking_date` – дата бронирования;
- `user_id` – идентификатор пользователя;
- `is_cancelled` – признак отмены бронирования.

Пример:

```
[
    {
        "id": 1,
```

```
        "room_id": 1,
        "booking_date": "2026-09-15",
        "user_id": 1,
        "is_cancelled": false
    }
]
```

В JSON хранятся именно идентификаторы связанных объектов, а не сами объекты Room и User.

При загрузке бронирования необходимо:

1) получить `room_id` из JSON;
2) найти соответствующий объект `Room` в коллекции `rooms`;
3) получить `user_id` из JSON;
4) найти соответствующий объект `User` в коллекции `users`;
5) передать найденные объекты в конструктор `Booking`.

Пример:

```
room = find_room_by_id(
    rooms,
    data["room_id"],
)

user = find_user_by_id(
    users,
    data["user_id"],
)

booking = Booking(
    booking_id=data["id"],
    room=room,
    booking_date=data["booking_date"],
    user=user,
)
booking.is_cancelled = data["is_cancelled"]
```

В результате объект `Booking` содержит ссылки на объекты `Room` и `User`:

```
Booking
├── id
├── room ──→ Room
├── booking_date
├── user ──→ User
└── is_cancelled
```

При сохранении выполняется обратное преобразование: вместо объектов `Room` и `User` в JSON записываются их идентификаторы.

```
{
    "id": booking.id,
    "room_id": booking.room.id,
    "booking_date": booking.booking_date,
    "user_id": booking.user.id,
    "is_cancelled": booking.is_cancelled,
}
```

Таким образом, принцип работы с данными остается следующим (рис. 8):

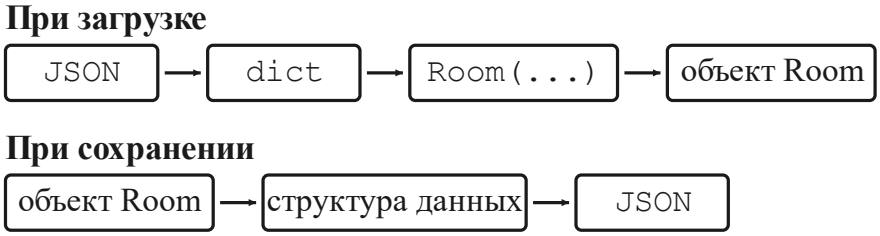


Рисунок 8 – Преобразование данных при загрузке и сохранении

Функции `load_rooms()`, `load_users()` и `load_bookings()` формируют коллекции объектов. Функции `save_rooms()`, `save_users()` и `save_bookings()` получают коллекции объектов и сохраняют их данные в JSON.

При загрузке `Booking` необходимо обеспечить наличие соответствующих объектов `Room` и `User`. Если помещение или пользователь с указанным идентификатором не найдены, бронирование не должно создаваться как корректный объект.

5.4.10. Адаптация автоматизированных тестов

После перехода к объектной модели тесты из ПР2 необходимо адаптировать. Проверяется не только результат выполнения функций, но и состояние объектов, их атрибуты и взаимодействие объектов между собой.

Шаг 1. Адаптация `test_rooms.py`

Проверить:

- создание объекта `Room`;
- значения его атрибутов;
- строковое представление объекта;
- проверку вместимости методом `is_suitable_for()`;
- поиск помещений;
- фильтрацию помещений;
- сортировку помещений;
- добавление помещения в коллекцию.

Пример:

```
from models import Room

from models.rooms import (
    add_room,
    check_room_capacity,
    find_room,
    sort_rooms,
    filter_rooms_by_capacity,
)

def test_room_creation():
    room = Room(
        1,
        "Аудитория 301",
        30,
    )

    assert room.id == 1
    assert room.name == "Аудитория 301"
    assert room.capacity == 30

def test_room_is_suitable_for():
    room = Room(
        1,
        "Аудитория 301",
        30,
    )

    assert room.is_suitable_for(20)
    assert not room.is_suitable_for(40)
```

Тесты функций поиска, фильтрации и сортировки необходимо адаптировать к коллекции объектов `Room`.

Шаг 2. Создание `test_users.py`

Создать отдельный файл:

```
tests/
├── test_rooms.py
├── test_users.py
└── test_bookings.py
```

Проверить:

- создание объекта User;
- значения его атрибутов;

- строковое представление;
- корректное создание пользователя из данных, если реализован метод from_data().

Пример:

```
from models import User

from models.users import (
    add_user,
    find_user,
   )

def test_user_creation():
    user = User(
        1,
        "Иван Петров",
        "ivan@example.com",
    )

    assert user.id == 1
    assert user.name == "Иван Петров"
    assert user.email == "ivan@example.com"
```

Шаг 3. Адаптация `test_bookings.py`

Для тестирования `Booking` необходимо предварительно создать объекты `Room` и `User`. Пример:

```
from models import Room

from models.rooms import (
    add_room,
    check_room_capacity,
    find_room,
    sort_rooms,
    filter_rooms_by_capacity,
)
def test_booking_creation():
    room = Room(
        1,
        "Аудитория 301",
        30,
    )
    user = User(
        1,
        "Иван Петров",
        "ivan@example.com",
    )
```

```
    booking = Booking(
        1,
        room,
        "2026-09-15",
        user,
    )

    assert booking.id == 1
    assert booking.room is room
    assert booking.user is user
    assert booking.booking_date == "2026-09-15"
```

Проверить отмену бронирования:

```
def test_booking_cancel():
    room = Room(
        1,
        "Аудитория 301",
        30,
    )

    user = User(
        1,
        "Иван Петров",
        "ivan@example.com",
    )

    booking = Booking(
        1,
        room,
        "2026-09-15",
        user,
    )

    booking.cancel()

    assert booking.is_cancelled
```

Шаг 4. Проверка правила доступности помещения

Одно из основных правил предметной области: нельзя создать второе активное бронирование одного и того же помещения на одну и ту же дату. При этом отменённое бронирование не должно блокировать помещение.

Например:

```
booking_1 = Booking(
    1,
    room,
```

```
    "2026-09-15",
    user,
)

booking_2 = create_booking(
    bookings,
    room,
    "2026-09-15",
    another_user,
)
```

Второе бронирование не должно быть создано, если первое остается активным. После отмены первого бронирования помещение снова может быть забронировано:

```
booking_1.cancel()
```

После этого создание нового бронирования на ту же дату должно быть разрешено.

Шаг 5. Сохранение тестов из ПР2

Тесты, проверявшие функциональность ПР2, не следует удалять без необходимости. Их необходимо адаптировать к объектной модели:

```
словари → объекты
room["capacity"] → room.capacity
room["name"] → room.name
booking["room_id"] → booking.room.id
booking["user_id"] → booking.user.id
```

Таким образом, переход к ООП не должен приводить к потере ранее реализованной функциональности.

Шаг 6. Запуск тестов

После адаптации всех тестов выполнить:

```
pytest -v
```

Результатом должно быть успешное выполнение всех тестов без ошибок.

5.5. Обновление README.md

После завершения рефакторинга необходимо обновить README.md, чтобы он соответствовал текущему состоянию проекта. Добавьте:

- описание основных классов;
- назначение классов;
- основные атрибуты;
- основные методы;
- описание взаимодействия объектов;
- обновленную структуру проекта;

- информацию о тестах;
- информацию о запуске программы.

Например:

```
# Roomly – система бронирования помещений

## Назначение

Roomly – консольное приложение для управления помещениями и их бронированием.

Приложение позволяет:

* просматривать помещения;
* находить помещения по названию;
* проверять вместимость помещений;
* проверять доступность помещения на определённую дату;
* создавать бронирования;
* отменять бронирования;
* просматривать список бронирований;
* добавлять новые помещения;
* сортировать помещения;
* хранить данные между запусками приложения.

## Целевая аудитория
Пользователи, которым необходимо бронировать помещения для проведения занятий, встреч и мероприятий.

## Предметная область

Основными объектами предметной области являются:
* помещение;
* пользователь;
* бронирование.

Пользователь может создавать бронирования помещений на определённые даты.
Одно помещение не может иметь два активных бронирования на одну и ту же дату.
Отменённое бронирование не блокирует помещение и позволяет создать новое бронирование на эту дату.

## Основные сущности
### Room – помещение
Объект `Room` представляет помещение, доступное для бронирования.
Основные атрибуты:
* `id` – уникальный идентификатор помещения;
* `name` – название помещения;
* `capacity` – вместимость помещения.
Основные методы:
* `is_suitable_for()` – проверка соответствия помещения заданному количеству людей;
* `__str__()` – строковое представление помещения.

### User – пользователь
Объект `User` представляет пользователя системы.
Основные атрибуты:
* `id` – уникальный идентификатор пользователя;
* `name` – имя пользователя;
* `email` – адрес электронной почты.
Основные методы:
* `__str__()` – строковое представление пользователя;
* `from_data()` – создание объекта `User` из набора данных.

### Booking – бронирование
Объект `Booking` представляет бронирование помещения пользователем.
Основные атрибуты:
* `id` – уникальный идентификатор бронирования;
* `room` – объект `Room`;
* `booking_date` – дата бронирования;
* `user` – объект `User`;
* `is_cancelled` – признак отмены бронирования.
Основные методы:
* `cancel()` – отмена бронирования;
* `__str__()` – строковое представление бронирования.

## Основные операции
### Работа с помещениями
* добавление помещения;
* поиск помещения по названию;
* проверка вместимости;
* фильтрация помещений по вместимости;
* сортировка помещений;
* просмотр списка помещений.
### Работа с пользователями
```

```
* создание пользователя;
* поиск пользователя;
* просмотр информации о пользователе.
### Работа с бронированиями
* создание бронирования;
* проверка доступности помещения;
* отмена бронирования;
* просмотр списка бронирований.

При создании бронирования проверяется наличие другого активного бронирования этого же помещения на ту же дату.
## Хранение данных
Данные приложения хранятся в JSON-файлах:
* `data/rooms.json`;
* `data/users.json`;
* `data/bookings.json`.

При загрузке данные из JSON преобразуются в объекты `Room`, `User` и `Booking`.
При сохранении объекты преобразуются в данные JSON.

Для связи бронирования с помещением и пользователем в `bookings.json` используются их идентификаторы `room_id` и `user_id`.

## Структура проекта
```text
room_booking/
│
├── README.md
├── requirements.txt
├── main.py
├── storage.py
├── utils.py
│
├── models/
│   ├── __init__.py
│   ├── rooms.py
│   ├── bookings.py
│   └── users.py
│
├── data/
│   ├── rooms.json
│   ├── bookings.json
│   └── users.json
│
└── tests/
    ├── test_rooms.py
    ├── test_bookings.py
    └── test_users.py
```

Назначение модулей
`main.py`

Точка входа в приложение.
Отвечает за:
* запуск приложения;
* отображение меню;
* получение данных от пользователя;
* организацию пользовательских сценариев.

`rooms.py`
Содержит класс `Room` и функции работы с помещениями.

`users.py`
Содержит класс `User` и функции работы с пользователями.

`bookings.py`
Содержит класс `Booking` и функции работы с бронированиями.

`storage.py`
Отвечает за загрузку и сохранение данных.
Выполняет преобразование:
* JSON → объекты Python;
* объекты Python → JSON.

`utils.py`

Содержит вспомогательные функции:
* `input_int()` – безопасный ввод целого числа;
* `input_date()` – безопасный ввод даты.

`tests/`
```

````
Содержит автоматизированные тесты проекта.

## Запуск приложения
Для запуска приложения выполнить:
```text
python main.py
```
После запуска отображается главное меню.

## Тестирование
Для запуска автоматизированных тестов выполнить:
```text
pytest -v
```
Тесты проверяют:
* создание и работу объектов `Room`, `User` и `Booking`;
* методы классов;
* операции с помещениями;
* создание и отмену бронирований;
* связь бронирования с помещением и пользователем;
* правила доступности помещения.

## Проверка качества кода
Для проверки соответствия кода требованиям PEP 8 используется `flake8`.
Команда проверки:
```text
flake8 .
```

## Git
Исходный код проекта хранится в Git-репозитории.
Для проверки состояния репозитория:

```text
git status
```

## План развития
* разработка веб-приложения;
* подключение базы данных;
* разработка API;
* контейнеризация приложения;
* настройка CI/CD.
````

5.6. Проверка и фиксация результата

Перед завершением работы необходимо выполнить итоговую проверку работоспособности приложения, автоматизированных тестов и качества кода, а также зафиксировать результаты разработки в Git.

Перед выполнением каждого коммита необходимо указывать метку текущей практической работы. Для ПР3 используется метка **`PR3`**.

Метка должна стоять в начале сообщения коммита, например:

```
PR3: add Room class
PR3: update booking functions
PR3: fix imports
PR3: add tests for User
```

Такая форма позволяет однозначно определить, к какой практической работе относится каждый коммит.

5.6.1. Проверка программы

Запустите приложение:

```
python main.py
```

Проверьте основные операции:

- просмотр помещений;
- поиск помещения;
- проверка вместимости;
- просмотр пользователей;
- проверка доступности помещения;
- создание бронирования;
- просмотр бронирований;
- отмена бронирования.

При проверке убедитесь, что:

- помещение и пользователь корректно выбираются при создании бронирования;
- нельзя создать два активных бронирования одного помещения на одну дату;
- отменённое бронирование перестаёт блокировать помещение;
- данные корректно отображаются после выполнения операций.

5.6.2. Запуск тестов

Запустите тесты:

```
pytest -v
```

Все тесты должны завершиться успешно.

При наличии ошибок необходимо определить их причину, внести исправления и повторно запустить тесты.

5.6.3. Проверка качества кода

Выполните проверку кода:

```
flake8 .
```

Необходимо устранить обнаруженные замечания, связанные с:

- форматированием кода;
- соблюдением требований PEP 8;
- отсутствием или некорректным использованием аннотаций типов;
- другими нарушениями, выявленными средствами проверки.

После внесения исправлений повторно выполните проверку.

5.6.4. Проверка изменений Git

Проверьте состояние репозитория:

```
git status
```

Просмотрите внесённые изменения:

```
git diff
```

Убедитесь, что в коммит не включены лишние файлы, временные файлы и файлы среды разработки.

Добавьте необходимые изменения:

```
git add .
```

Создайте информативный коммит:

```
git commit -m "Refactor project to OOP"
```

При использовании удалённого репозитория отправьте изменения:

```
git pus
```

6. РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ

К завершению ПР3 должны быть получены следующие результаты. Используйте этот перечень для самопроверки перед переходом к контролю: каждый пункт должен быть подтвержден демонстрацией или содержимым репозитория.

Материалы Яндекс Практикума

Раздел 03. «ООП и другие полезные вещи, которые нужно знать. Объекты и классы»:

- Введение. ООП;
- Классы и объекты в Python;
- Собственные классы и объекты;
- Атрибуты класса и объекта;
- Методы объекта;
- Магический метод `__str__`;
- Практика по теме.

Раздел 03. «ООП и другие полезные вещи, которые нужно знать. Знакомство с ООП»:

- Принципы ООП;
- Наследование;
- Полиморфизм;
- Инкапсуляция;
- Практика по теме.

Раздел 03. «ООП и другие полезные вещи, которые нужно знать. Расширенные возможности Python»:

- Декораторы;
- Декораторы для методов класса.
- Практика по теме.

Раздел 03. «Финальный проект спринта: «Из-гиб Питона»:

- «Подготовка проекта к автотестам»
- «Проверка проекта автотестами»

- «Проект. Игра «Изгиб Питона»»

Материалы ПР1 и ПР2 в рамках данной работы повторно не выполняются, но продолжают использоваться в индивидуальном проекте.

Индивидуальный проект

- проект ПР2 сохранен и продолжен;
- основные сущности предметной области представлены классами;
- созданы объекты классов;
- определены атрибуты;
- реализованы методы;
- используется `__init__`;
- реализовано строковое представление `__str__`;
- организовано взаимодействие объектов;
- коллекции содержат объекты предметной области;
- функции ПР2 адаптированы к новой объектной модели;
- сохранена основная функциональность проекта;
- данные продолжают храниться в JSON;
- загрузка JSON создает объекты;
- сохранение объектов в JSON работает корректно;
- сохранена обработка исключений;
- автоматизированные тесты адаптированы и проходят;
- код проверен flake8;
- основные функции и методы имеют аннотации типов;
- обновлен `README.md`;
- результат зафиксирован в Git.

7. ФОРМА КОНТРОЛЯ И КРИТЕРИИ ОЦЕНКИ

Контроль выполнения ПР3 включает:

- проверку материалов Яндекс Практикума;
- демонстрацию индивидуального проекта;
- проверку структуры репозитория;
- проверку объектной модели;
- проверку работоспособности программы;
- проверку автоматизированных тестов;
- проверку качества кода;
- ответы на контрольные вопросы.

7.1. Чек-лист выполнения материалов Яндекс Практикума

Студент предъявляет преподавателю результат выполнения следующих материалов. Отмечайте пункты по мере выполнения – чек-лист используется и при самопроверке, и при контроле со стороны преподавателя.

Чек-лист разделов «Объекты и классы» и «Знакомство с ООП»

№	Материал	Выполнено
1	Введение. ООП	☐
2	Классы и объекты в Python	☐
3	Собственные классы и объекты	☐
4	Атрибуты класса и объекта	☐
5	Методы объекта	☐
6	Магический метод __str__	☐
7	Практика по теме	☐
8	Принципы ООП	☐
9	Наследование	☐
10	Полиморфизм	☐
11	Инкапсуляция	☐
12	Практика по теме	☐

Чек-лист раздела «Расширенные возможности Python»

№	Материал	Выполнено
1	Декораторы	☐
2	Декораторы для методов класса	☐
3	Практика по теме	☐

Чек-лист раздела «Финальный проект спринта: «Изгиб Питона»»

№	Материал	Выполнено
1	Подготовка проекта к автотестам	☐
2	Проверка проекта автотестами	☐
3	Проект. Игра «Изгиб Питона»	☐

7.2. Чек-лист выполнения индивидуального проекта

Отмечайте пункты по мере выполнения – чек-лист используется при самопроверке и при контроле со стороны преподавателя.

№	Требование	Выполнено
1	Проект ПР2 сохранен и продолжен	☐
2	Основные сущности представлены классами	☐
3	Создаются объекты классов	☐
4	Определены атрибуты объектов	☐
5	Реализованы методы	☐
6	Используется `__init__`	☐
7	Реализован `__str__`	☐
8	Организовано взаимодействие объектов	☐
9	Коллекции содержат объекты	☐
10	Функции ПР2 адаптированы	☐
11	Основная функциональность сохранена	☐
12	Данные хранятся в JSON	☐

№	Требование	Выполнено
13	JSON корректно преобразуется в объекты	☐
14	Объекты корректно сохраняются в JSON	☐
15	Сохранена обработка исключений	☐
16	Автоматизированные тесты адаптированы	☐
17	Все тесты проходят	☐
18	flake8 не обнаруживает ошибок	☐
19	Основные функции и методы имеют аннотации типов	☐
20	`README.md` обновлен	☐
21	Результат зафиксирован в Git	☐

7.3. Критерии оценки

Работа считается выполненной, если по каждому показателю достигнут требуемый результат. Критерии используются преподавателем при защите результата работы.

№	Критерий	Содержание
1	Материалы Практикума	Выполнены предусмотренные материалы ПР3
2	Классы и объекты	Основные сущности представлены классами, объекты создаются корректно
3	Атрибуты и методы	Атрибуты и поведение распределены между объектами обоснованно
4	`__str__`	Реализовано удобное строковое представление объектов
5	Взаимодействие объектов	`User`, `Booking` и `Room` взаимодействуют в рамках предметной области
6	Переработка проекта	Переход от функциональной модели ПР2 к объектной выполнен последовательно
7	Хранение данных	JSON-загрузка и сохранение работают с объектной моделью
8	Принципы ООП	Продемонстрировано понимание инкапсуляции, наследования и полиморфизма
9	Тестирование	Тесты адаптированы и проходят
10	Качество кода	Соблюдены требования к структуре и стилю кода
11	Документация	README отражает актуальное состояние проекта
12	Git	Результат работы зафиксирован в репозитории

Декораторы оцениваются прежде всего в рамках понимания материала Яндекс Практикума. Их применение в индивидуальном проекте не является обязательным, если для конкретной задачи нет обоснованной необходимости.

7.4. Требования к отчету

Объем отчета – 1-2 страницы, не считая титульной информации. Отчет должен содержать:

1. *Общие сведения*:

- название практической работы;
- ФИО;
- группа;
- название индивидуального проекта.

2. *Выполненные материалы Яндекс Практикума*:

Раздел	Выполненные темы
Раздел 03. «ООП и другие полезные вещи, которые нужно знать»	Введение в ООП; классы и объекты; атрибуты и методы; __str__; принципы ООП; наследование; полиморфизм; инкапсуляция
Раздел 03. «Расширенные возможности Python»	Декораторы; декораторы для методов класса
Раздел 03. «Финальный проект спринта: «Изгиб Питона»»	Подготовка проекта к автотестам. Проверка проекта автотестами. Проект. Игра «Изгиб Питона»

3. *Доработка индивидуального проекта*. Указать:

- название проекта;
- основные классы, добавленные или переработанные на ПР3;
- краткое описание выполненной доработки;
- ссылку на Git-репозиторий.

4. *Результат работы.* Кратко указать, что в результате выполнения ПР3:

- основные сущности проекта представлены классами и объектами;
- реализованы атрибуты и методы классов;
- организовано взаимодействие объектов;
- существующая функциональность проекта сохранена и адаптирована к объектной модели;
- обновлены тесты и документация проекта.

5. *Вывод.* Сделать краткий вывод о результатах выполнения практической работы.

8. КОНТРОЛЬНЫЕ ВОПРОСЫ

При подготовке к защите используйте контрольные вопросы. Они сгруппированы по четырем направлениям и охватывают все разделы работы: классы и объекты, принципы ООП, методы и декораторы, индивидуальный проект. Ответ должен сопровождаться демонстрацией соответствующего участка кода проекта.

По классам и объектам

1. Что такое класс?
2. Что такое объект?
3. Чем класс отличается от объекта?
4. Что такое экземпляр класса?
5. Для чего используется `__init__()`?
6. Что такое `self`?
7. Чем атрибут класса отличается от атрибута объекта?
8. Что такое метод объекта?
9. Чем метод отличается от обычной функции?

10. В каких случаях функцию следует сделать методом класса?
11. Для чего используется `__str__()`?
12. Что происходит при выполнении `print(object)`?
13. Что такое конструктор объекта?
14. Можно ли создать несколько объектов одного класса?
15. Могут ли объекты одного класса иметь разные значения атрибутов?

По принципам ООП

16. Что такое инкапсуляция?
17. Как инкапсуляция проявляется в проекте?
18. Что такое наследование?
19. Что означает отношение «является» при наследовании?
20. Почему не следует использовать наследование без предметного основания?
21. Что такое полиморфизм?
22. Как `__str__()` связан с полиморфизмом?
23. Что такое композиция?
24. Чем композиция отличается от наследования?
25. Как композиция используется в классе `Booking`?

По методам и декораторам

26. Что такое декоратор?
27. Для чего используется декоратор?
28. Что такое функция-обертка?
29. Для чего используются `*args` и `**kwargs` в декораторах?
30. Для чего используется `functools.wraps`?
31. Для чего применяется `@staticmethod`?
32. Чем `staticmethod` отличается от обычного метода?
33. Для чего используется `@classmethod`?
34. Чем `classmethod` отличается от метода объекта?
35. Для чего используется `@property`?
36. Почему `booking.status` может использоваться без круглых скобок?

По индивидуальному проекту

37. Какие сущности выделены в индивидуальном проекте?
38. Почему `Room` представлен классом?
39. Какие атрибуты имеет `Room`?

40. Какие операции относятся непосредственно к `Room`?

41. Почему `Booking` хранит объект `Room`, а не только его идентификатор?

42. Почему `Booking` связан с объектом `User`?

43. Какие функции ПР2 были преобразованы в методы?

44. Какие функции ПР2 остались обычными функциями и почему?

45. Как выполняется преобразование JSON в объекты?

46. Как выполняется сохранение объектов в JSON?

47. Как изменились коллекции данных по сравнению с ПР2?

48. Какие тесты были адаптированы после перехода к ООП?

49. Как проверить, что старая функциональность проекта не была потеряна?

50. Какова роль main.py после перехода к объектной модели?

9. ИТОГ

В ПР3 функциональный проект из ПР2 переработан в объектно-ориентированную модель. В результате:

- основные сущности предметной области представлены классами;
- созданы объекты;
- данные и поведение объединены в объектах;
- использованы атрибуты и методы;
- реализованы `__init__()` и `__str__()`;
- организовано взаимодействие объектов;
- рассмотрены инкапсуляция, наследование и полиморфизм;
- коллекции содержат объекты предметной области;
- существующие функции проекта переработаны с учетом объектной модели;
- сохранена работа с JSON-файлами;
- сохранены тестирование, обработка исключений и контроль качества кода;
- изученные декораторы.

Индивидуальный проект после ПР3 имеет полноценную объектную модель и готов к дальнейшему переходу от консольного приложения к веб-приложению.

Следующий этап – использование объектной модели при разработке приложения на **Django**.